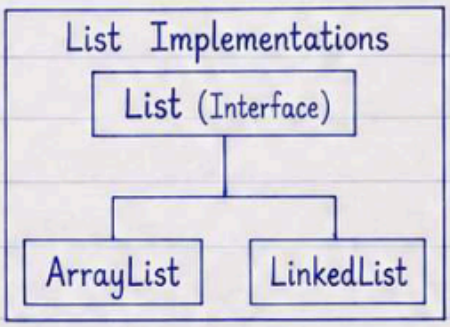
Java Collections - List

1. What is List ?

- List is an ordered collection (maintains insertion order).
- Allows duplicate elements.



2. Creating a List

```
List<String> list = new ArrayList<>();      // Creates an empty ArrayList
List<Integer> list = new LinkedList<>();    // Creates an empty LinkedList
```

3. Common List Methods

List.add(element)	:	Adds an element to the end of the list.
List.add(index, element)	:	Adds an element at the specified index.
list.get(index)	:	Returns element at the specified index.
list.set(index, element)	:	Updates element at the specified index.
list.remove(index)	:	Removes element at the specified index.
list.remove(element)	:	Removes first occurrence of the element.
list.contains(element)	:	Checks if the element exists in the list.
list.size()	:	Returns total number of elements.
list.isEmpty()	:	Checks if the list is empty.

4. Iterating a List

```
// Using for loop
for (int i = 0; i < list.size(); i++) {
    System.out.println(list.get(i));
}

// Using enhanced for loop
for (String s : list) {
    System.out.println(s);
}
```

5. ArrayList vs LinkedList

	ArrayList	LinkedList
Based on	Dynamic array	Doubly linked list
Access time	Faster (O(1))	Slower (O(n))
Insertion/Deletion	Slower (O(n))	Faster (O(1))
Memory	Less	More
Best used for	Frequent read	Frequent insert/delete

Java Collections - Set

1. What is Set ?

- Set is a collection that does not allow duplicate elements.
- It does not maintain insertion order (except LinkedHashSet).
- Part of Set interface.

2. Creating a Set

```
Set<String> set = new HashSet<>();          // Unordered set
Set<Integer> set = new LinkedHashSet<>();   // Maintains insertion order
Set<Integer> set = new TreeSet<>();         // Sorted set (natural order)
```

3. Common Set Methods

```
set.add(element)       : Adds an element to the set.
set.remove(element)    : Removes the specified element.
set.contains(element)  : Checks if the element exists.
set.size()             : Returns total number of elements.
set.isEmpty()          : Checks if the set is empty.
set.clear()            : Removes all elements from the set.
```

Example

```
Set<Integer> set =
  new HashSet<>();
set.add(1);
set.add(2);
set.add(2);  // duplicate
System.out.println(set);
// Output: [1, 2]
```

4. Types of Set

- HashSet : No guaranteed order, allows null, fastest.
- LinkedHashSet : Maintains insertion order, allows null.
- TreeSet : Sorted order (natural ordering), does not allow null.

5. HashSet vs LinkedHashSet vs TreeSet

Feature	HashSet	LinkedHashSet	TreeSet
Order	No guaranteed order	Insertion order	Sorted order
Null allowed	Yes (one null)	Yes (one null)	No
Performance	Faster (O(1))	Slightly slower (O(1))	Slower (O(log n))
Internal structure	Hash table	Hash table + Linked list	Red-Black Tree
Use case	Unique elements	Unique + maintain order	Unique + sorted elements

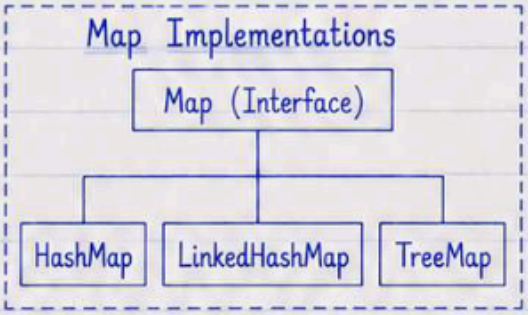
Note :

- Set is useful when you want to store only unique elements.
- Use TreeSet when you need elements in sorted order.

Java Collections - Map

1. What is Map ?

- Map is a collection of key-value pairs.
- Keys are unique, values can be duplicate.
- Map does not maintain insertion order (except LinkedHashMap).
- Part of Map interface.



2. Creating a Map

```
Map<Integer, String> map = new HashMap<>();        // Unordered map
Map<Integer, String> map = new LinkedHashMap<>();  // Maintains insertion order
Map<Integer, String> map = new TreeMap<>();        // Sorted by keys (natural order)
```

3. Common Map Methods

map.put(key, value)	:	Adds a key-value pair.
map.get(key)	:	Returns value for the key.
map.getOrDefault(key, value)	:	Returns value if key exists, otherwise default value.
map.containsKey(key)	:	Checks if key exists.
map.containsValue(value)	:	Checks if value exists.
map.remove(key)	:	Removes the entry for the key.
map.size()	:	Returns total number of key-value pairs.
map.keySet()	:	Returns a set of all keys.
map.values()	:	Returns a collection of all values.
map.entrySet()	:	Returns a set of key-value pairs (Map.Entry).

Example

```
Map<Integer, String> map =
    new HashMap<>();
map.put(101, "Aman");
map.put(102, "Ravi");
map.put(103, "Neha");

System.out.println(map.get(102));
System.out.println(map.keySet());
System.out.println(map.values());
// Output:
// Ravi
// [101, 102, 103]
// [Aman, Ravi, Neha]
```

4. Types of Map

- HashMap : No guaranteed order, allows one null key and multiple null values.
- LinkedHashMap : Maintains insertion order, allows one null key and multiple null values.
- TreeMap : Sorted by keys (natural order), does not allow null keys (null values allowed).

5. HashMap vs LinkedHashMap vs TreeMap

Feature	HashMap	LinkedHashMap	TreeMap
Order	No guaranteed order	Insertion order	Sorted order (natural)
Null key	One null key allowed	One null key allowed	Not allowed
Null values	Multiple allowed	Multiple allowed	Multiple allowed
Performance	Faster (O(1))	Slightly slower (O(1))	Slower (O(log n))
Use case	General purpose	When order is important	When sorted keys are needed

Note :

- Use HashMap for most of the cases.
- Use LinkedHashMap when you want to maintain insertion order.
- Use TreeMap when you need keys in sorted order.

Java Collections - Queue & Utility

1. What is Queue ?

- Queue is a collection that follows FIFO (First In First Out).
- Elements are added at the rear and removed from the front.
- Part of Queue interface.

Queue Implementation



2. Common Queue Methods

queue.offer(element)	:	Adds an element to the queue (returns false if fails).
queue.poll()	:	Removes and returns the head element (returns null if empty).
queue.peek()	:	Returns the head element without removing it.
queue.size()	:	Returns total number of elements.
queue.isEmpty()	:	Checks if the queue is empty.

Example (PriorityQueue)

```
Queue<Integer> pq =
  new PriorityQueue<>();
pq.offer(30);
pq.offer(10);
pq.offer(20);
System.out.println(pq.poll()); // 10
System.out.println(pq.peek()); // 20
```

3. Types of Queue

Type	Description	Order	Null Allowed	Use Case
PriorityQueue	Elements processed based on priority	Priority order (smallest first)	No	Scheduling, heap, top k elements
Deque (ArrayDeque)	Double ended queue	Insertion at both ends	No	Use as Queue or Stack
LinkedList	Implements Queue & Deque	FIFO	Yes	General purpose

4. Useful Methods in Collections Class

Collections.sort(list)	:	Sorts the list in ascending order.
Collections.sort(list, reverseOrder())	:	Sorts the list in descending order.
Collections.reverse(list)	:	Reverses the elements in the list.
Collections.shuffle(list)	:	Randomly shuffles the elements.
Collections.max(list)	:	Returns the maximum element.
Collections.min(list)	:	Returns the minimum element.
Collections.frequency(list, element)	:	Returns the number of occurrences.
Collections.copy(dest, src)	:	Copies elements from src to dest.
Collections.fill(list, element)	:	Fills the list with the specified element.

Example

```
List<Integer> list =
   new ArrayList<>();
list.add(5);
list.add(2);
list.add(8);
Collections.sort(list);
System.out.println(list);   // [2, 5, 8]
Collections.reverse(list);
System.out.println(list);   // [8, 5, 2]
System.out.println(Collections.max(list)); //8
```

5. When to Use What ?

Collection	Use When	Key Points
List	You need ordered data (duplicates allowed)	ArrayList (fast), LinkedList (frequent insert/delete)
Set	You need unique elements	HashSet, LinkedHashSet, TreeSet
Map	You need key-value pairs	HashMap, LinkedHashMap, TreeMap
Queue	You need FIFO or priority-based processing	PriorityQueue, Deque

Java Collections - Comparable & Comparator

1. Comparable Interface

- Used for natural/default sorting.
- Class must implement Comparable<T>.
- Defines compareTo() method.
- Sorting is based on single logic.

```
class Employee implements Comparable<Employee> {
    int id;
    String name;
    public int compareTo(Employee e) {
        return this.id - e.id;   // sort by id
    }
}

Collections.sort(list);  // uses compareTo()
```

2. Comparator Interface

- Used for custom sorting.
- Does not modify the original class.
- Defines compare(T o1, T o2) method.
- Allows multiple sorting logics.

```
Comparator<Employee> salaryComparator =
    new Comparator<Employee>() {
    public int compare(Employee e1, Employee e2) {
        return e1.salary - e2.salary;  // sort by salary
    }
};
```

Using Lambda (Java 8+)

```
Comparator<Employee> comp =
    Comparator.comparing(Employee::getSalary);
list.sort(comp);  // sort by salary

list.sort(Comparator.comparing(Employee::getName)
        .thenComparing(Employee::getId));
```

3. Comparable vs Comparator

Feature	Comparable	Comparator
Method	compareTo(T o)	compare(T o1, T o2)
Package	java.lang	java.util
Sorting type	Natural/default sorting	Custom sorting
Implementation	Inside the class	Separate class / Lambda
Number of sorting logics	Only one	Multiple
Changes in class	Modifies the class	No changes required
Use case	When natural order is enough	When different sorting criteria are needed

4. Iterator and ListIterator

- Iterator is used to traverse a collection.
- ListIterator is used for List (bi-directional).
- Iterator supports remove().
- ListIterator supports both forward and backward traversal.

Iterator Methods

```
hasNext()   : Checks if next element exists.
next()      : Returns next element.
remove()    : Removes last returned element.
```

ListIterator Methods

```
hasNext()      : Checks if next element exists.
hasPrevious()  : Checks if previous element exists.
next()         : Returns next element.
previous()     : Returns previous element.
add(element)   : Adds element at current position.
remove()       : Removes last returned element.
```

5. Example - Sorting a List

```
List<Employee> list = new ArrayList<>();
list.add(new Employee(3, "Ravi", 50000));
list.add(new Employee(1, "Aman", 80000));
list.add(new Employee(2, "Neha", 60000));

Collections.sort(list);  // uses Comparable (id)
System.out.println(list);

list.sort(Comparator.comparing(Employee::getSalary));
System.out.println(list);  // sort by salary
```

6. Important Points

- Collections.sort(list) uses natural ordering (Comparable).
- Use Comparator for custom or multiple sorting.
- Comparator.comparing() and thenComparing() are useful in Java 8+.
- Iterators are fail-fast (ConcurrentModificationException) if collection is modified during iteration.
- Use ListIterator when you need to traverse in both directions or modify the list while iterating.